

PRÁCTICA · RA1

Automatización de un problema mediante Python

Lectura, validación y procesamiento reutilizable de datasets

M3 · Programació d'Intel·ligència Artificial

RA1 · Desarrollo de programas de inteligencia artificial

Práctica individual · 3–4 horas

Curso de Especialización en Inteligencia Artificial y Big Data

Índice

Contexto	3
Objetivo de la práctica	3
Definición del problema	3
Ejemplo orientativo	3
Datasets	3
Lectura de datos	4
Validación de los datos	4
Procesamiento	4
Funciones y modularidad	5
Flujo principal	5
Generación de resultados	5
Pruebas obligatorias	6
Restricciones	6
Entrega	6
Criterios de evaluación	6

1. Contexto

En muchos entornos profesionales existen tareas repetitivas que consisten en recibir datos, comprobarlos, aplicar una serie de reglas y generar un resultado.

Algunos ejemplos podrían ser:

- revisar un inventario y detectar productos con poco stock;
- analizar notas de estudiantes y generar un resumen;
- procesar incidencias y clasificarlas por prioridad;
- revisar datos de sensores y detectar valores fuera de rango;
- analizar pedidos y obtener estadísticas;
- organizar una colección de películas, libros u otros registros;
- procesar registros de asistencia;
- revisar gastos y clasificarlos por categoría.

En esta práctica deberás identificar tu propio problema susceptible de ser automatizado y desarrollar una pequeña aplicación en Python que lo resuelva.

Importante: no se utilizarán algoritmos de Machine Learning. El objetivo es diseñar correctamente un programa, trabajar con archivos y construir una solución reutilizable.

2. Objetivo de la práctica

El objetivo es desarrollar un programa capaz de:

- 1 leer datos procedentes de distintos archivos;
- 2 validar y normalizar la información;
- 3 aplicar reglas de procesamiento;
- 4 generar resultados útiles;
- 5 funcionar con más de un dataset sin modificar su lógica principal;
- 6 organizar el código mediante funciones y módulos.

3. Definición del problema

Antes de empezar a programar deberás definir el problema que quieres resolver.

El problema debe representar una tarea que tenga sentido automatizar porque pueda repetirse sobre diferentes conjuntos de datos.

Deberás explicar brevemente:

- qué situación quieres automatizar;
- qué datos necesita el programa;
- qué recibe como entrada;
- qué operaciones realizará;
- qué resultado debe producir.

Ejemplo orientativo

Una tienda recibe periódicamente archivos con información de su inventario. Se quiere desarrollar un programa que revise automáticamente los productos, detecte aquellos con poco stock, calcule el valor total del inventario y genere un informe.

El ejemplo es únicamente orientativo. Cada alumno deberá proponer su propio caso.

4. Datasets

El programa deberá poder trabajar, como mínimo, con dos datasets diferentes relacionados con el problema elegido.

Además, deberán utilizarse al menos estos dos formatos de entrada:

- CSV
- JSON

Por ejemplo:

```
inventario_tienda_A.csv
inventario_tienda_B.json
```

Los dos archivos no tienen que tener exactamente la misma representación interna, pero deberán contener la información necesaria para resolver el mismo problema.

La aplicación deberá convertir ambos formatos a una estructura interna común antes de procesarlos.

```
Flujo de trabajo
CSV → lectura → normalización → procesamiento\ JSON
```

5. Lectura de datos

La aplicación deberá detectar o permitir indicar el tipo de archivo utilizado y cargar correctamente sus datos.

Como mínimo deberán existir funciones diferenciadas para la lectura:

```
def cargar_csv(ruta):
    ...

def cargar_json(ruta):
    ...
```

Ambas funciones deberán generar una estructura compatible con el resto del programa.

6. Validación de los datos

Los datasets deberán incluir algunos datos problemáticos para comprobar que la aplicación es capaz de gestionarlos.

El programa deberá validar, como mínimo:

- presencia de los campos obligatorios;
- valores vacíos;
- tipos de datos incorrectos;
- valores que no tengan sentido según el problema.

Ejemplos:

```
stock = -25
edad = "treinta"
precio = ""
temperatura = 900
```

El programa no debe simplemente finalizar con un error ante un dato incorrecto.

Deberás decidir qué hacer en cada situación:

- ignorar el registro;
- corregirlo cuando sea posible;
- convertir el valor;
- marcarlo como inválido;
- mostrar un aviso.

Las decisiones tomadas deberán estar justificadas en el README.

7. Procesamiento

El programa deberá realizar al menos cuatro operaciones útiles relacionadas con el problema elegido.

Algunas posibilidades son:

- calcular estadísticas;
- clasificar registros mediante reglas;
- filtrar elementos;
- buscar valores;

- contar categorías;
- detectar situaciones concretas;
- ordenar resultados;
- generar indicadores.

No se trata de cumplir artificialmente cuatro operaciones, sino de construir una aplicación que produzca información útil.

8. Funciones y modularidad

El código no podrá consistir en un único bloque de instrucciones.

La aplicación deberá estar dividida en funciones con responsabilidades claramente diferenciadas.

Una posible estructura sería:

```
def cargar_datos(...):  
    ...  
  
def validar_datos(...):  
    ...  
  
def limpiar_datos(...):  
    ...  
  
def procesar_datos(...):  
    ...  
  
def generar_resumen(...):  
    ...  
  
def guardar_resultados(...):  
    ...
```

No es obligatorio utilizar exactamente estas funciones. La estructura deberá adaptarse al problema elegido.

9. Flujo principal

El programa deberá disponer de un flujo de ejecución fácilmente comprensible.

Por ejemplo:

```
Flujo de trabajo  
Seleccionar archivo → Detectar formato → Cargar datos \\ \\ Validar → Procesar → Mostrar resultados → Guardar  
resultado
```

El usuario deberá poder ejecutar el programa con cualquiera de los datasets preparados sin modificar manualmente el código interno.

10. Generación de resultados

La aplicación deberá generar al menos un archivo de salida.

Se puede utilizar:

- JSON;
- CSV;
- TXT.

El resultado dependerá del problema planteado.

Ejemplo:

```
{  
  "registros_procesados": 250,  
  "registros_invalidos": 7,  
  "productos_stock_bajo": 18,  
  "valor_total_inventario": 12453.70  
}
```

11. Pruebas obligatorias

La aplicación deberá probarse como mínimo con:

- Dataset 1: datos principalmente correctos.
- Dataset 2: datos con diferencias o problemas que obliguen a aplicar los mecanismos de validación.

Además, deberán comprobarse al menos tres situaciones problemáticas, por ejemplo:

- archivo vacío;
- campo obligatorio inexistente;
- valor numérico incorrecto;
- registro duplicado;
- categoría desconocida.

12. Restricciones

No se permite:

- escribir manualmente dentro del código los resultados esperados;
- modificar el programa específicamente para que funcione únicamente con un dataset;
- crear una única función que realice todo el proceso;
- eliminar todos los registros problemáticos sin justificar la decisión;
- utilizar Machine Learning.

Se valorará especialmente que el programa pueda reutilizarse con nuevos datasets compatibles sin modificar su lógica principal.

13. Entrega

Se entregará un único archivo comprimido con una estructura similar a:

```
apellido_nombre_RA1_automatizacion/
|
|-- main.py
|-- funciones.py
|-- datos/
|   |-- dataset_1.csv
|   |-- dataset_2.json
|-- resultados/
|   |-- resultado.json
|-- README.md
```

El archivo README.md deberá explicar, de forma breve:

- 1 problema elegido;
- 2 datos utilizados;
- 3 funcionamiento general del programa;
- 4 reglas de validación;
- 5 decisiones tomadas;
- 6 instrucciones para ejecutar la aplicación.

La extensión recomendada del README es de 1–2 páginas equivalentes.

14. Criterios de evaluación

1.25

Apartado	Peso
Definición y adecuación del problema	10%
Lectura y gestión de CSV y JSON	15%
Validación y gestión de errores	20%

Apartado	Peso
Procesamiento y utilidad de los resultados	20%
Funciones, modularidad y estructura del código	20%
Pruebas con distintos datasets	10%
README y claridad de la entrega	5%
Total	100%

Importante

Requisito mínimo: el mismo programa deberá funcionar con los dos datasets entregados sin reescribir su lógica principal.